tensortools: Statistical Modeling and Computation for Tensor-Valued Data

Anupama Kannan David Kahle Michael Gallaugher

26-10-19

tensortools is an R package for simulation, maximum likelihood estimation, and model diagnostics for tensor-variate distributions arising in multiway array data. The package implements the tensor-variate normal distribution and several normal variance–mean mixture families, including the skew-t, generalized hyperbolic, variance gamma, and normal inverse Gaussian distributions. Estimation is performed via likelihood-based and expectation–maximization algorithms under a Kronecker-separable covariance model, yielding mode-specific covariance estimates through iterative updates. Model assessment tools include comparisons of Mahalanobis distances computed from separable and unrestricted covariance estimates. In addition, tensortools provides implementations of widely used tensor decompositions for approximation and dimension reduction, including Tensor-Train decomposition via TT-SVD, Higher-Order Singular Value Decomposition (HOSVD), and Parallel Factor Analysis (CP/PARAFAC) computed using alternating least squares. The package also includes common linear algebra and tensor operations, such as matricization, the n-mode products, and the Khatri-Rao product.

Introduction

Modern datasets increasingly have meaningful multiway structure. This is due to new technologies and an interest in analyzing relationships in large, high-dimensional datasets. Tensor models can preserve multiway structure, leading to more interpretability in the results. A common approach is to vectorize or matricize this higher order data to fit pre-established models and tools. Applying vectorized models ignores the original covariance structure of the data, which can require estimating many more parameters compared to assuming there is a separable covariance structure. Additionally, many researchers assume that data is normal. This can lead to models that fit poorly when observations are skewed or heavy-tailed.

The package `tensortools` aims to assist with both of these issues. It contains the tensor variate normal distribution and several skewed distributions under the normal variance-mean

mixture family. We use the flip-flop algorithm for the tensor-variate normal and expectation-maximization algorithm for the normal variance-mean mixtures to compute the maximum likelihood estimators for the user's dataset. Diagnostic and goodness-of-fit tests support model assessment and comparison. It also includes random number generation and density functions for researchers who want to better understand tensor variate distributions.

In the following sections, we will explain the theory behind tensor variate distributions, compare related tensor software across several programming languages, explain some design choices in the package, show a simulation study, and apply the models to a real-world problem.

Tensor notation

Tensor notation varies across disciplines, so we begin by defining the conventions and operations used in `tensortools`. Tensors are a generalization of vectors and matrices, allowing us to represent multi-way data with a specific structure.

Order

The order of a tensor is the number of indices needed to identify an entry. This is the same as the number of values needed inside R's subsetting brackets to pull one element out. A vector is a first-order tensor, a matrix is a second-order tensor, and any object with three or more modes is a higher-order tensor.

For example, take the vector $\mathbf{a} = (1, 2, 3)$. To extract a specific element, you only require one index, so it is an order-one object.

```
a <- c(1, 2, 3)

a[2]
#> [1] 2
```

The matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

has order two, since its entries are indexed by row and column.

```
A <- matrix(data = 1:6, nrow = 2, ncol = 3)

A[2, 1]
#> [1] 2
```

A third-order tensor requires three indices. The following tensor has dimensions $4 \times 3 \times 2$, so we need to supply an index for each mode to extract an element.

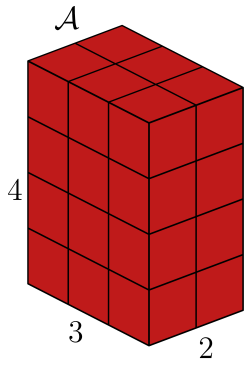


Figure 1: A $4 \times 3 \times 2$ tensor $\mathcal{A}$

```
A <- tensor(data = 1:24, dim = c(4, 3, 2))

A[3, 1, 1]
#> [1] 3
```

Mode

A mode is a specific axis of a tensor. For a matrix, the two modes are usually called rows and columns. However, since a tensor can have three or more dimensions, we now need a term for the specific dimension being referred to.

Consider a tensor $\mathcal{A} \in \mathbb{R}^{4\times2\times3}$. We would say the first mode has a size of 4, the second mode has a size of 2, and the third mode has a size of 3. Note that the modes of a tensor are dependent on the structure of the object. For example, we could keep the same data in $\mathcal{A}$ but permute the modes, leading to a new tensor $\mathcal{B} \in \mathbb{R}^{4\times3\times2}$. This changes the arrangement and interpretation of entries, even though the values themselves are the same. When communicating a tensor, it is important to keep the modes consistent and keep track of what each mode represents. For example, the modes of $\mathcal{A}$ could represent a sensor ID, time, and location respectively.

n-mode product

In regular matrix multiplication, we can multiply two matrices as long as the number of columns in the first matrix matches the rows of the second matrix. Suppose we want to multiply a tensor and a matrix. We need a new operator to specify which mode from the tensor should line up with the matrix. The n-mode product specifies that the n-th mode of a tensor should have the same size as the columns of the matrix.

Let tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_D}$ and let a matrix $\mathbf{U} \in \mathbb{R}^{J \times I_n}$. That is, the number of columns in $\mathbf{U}$ must have the same size as the n-th mode of $\mathcal{A}$. Then the n-mode product of $\mathcal{A}$ and $\mathbf{U}$ can be defined as

$$(\mathcal{A} \times_n \mathbf{U}) \in \mathbb{R}^{I_1 \times \ldots \times I_{n-1} \times J \times I_{n+1} \times \ldots \times I_D}.$$

Note that the modes of the result stay the same as $\mathcal{A}$ other than the n-th mode being replaced with J. The order of the result is the same as the order of the input tensor $\mathcal{A}$. Its entries are

$$(\mathcal{A} \times_n \mathbf{U})_{i_1,\ldots,i_{n-1},j,i_{n+1},\ldots i_D} = \sum_{i_n=1}^{I_n} a_{i_1,\ldots,i_n,\ldots,i_D} u_{j,i_n}$$

The tensor and matrix have a mode of the same size, denoted here as I_n. This operation is also called the tensor times matrix product.

This function is called `n_prod()` in the package `tensortools`. It accepts a tensor, a value n specifying which mode to multiply across, and a matrix. For example, let us take $\mathbf{a} \in \mathbb{R}^3$ and $\mathbf{U} \in \mathbb{R}^{2 \times 3}$. Then if we compute $\mathbf{a} \times_1 \mathbf{U}$, we expect the final result to lie $\in \mathbb{R}^2$.

```
a_vec <- c(1, 2, 3)
u_mat <- matrix(4:9, ncol = 3)

n_prod(tensor = a_vec, n = 1, mat = u_mat)
#> [1] 40 46
```

This function is supported as an infix operator with the notation `A %xn% (U~n)`, where `%xn%` is an infix operator and `n` is the mode computed across.

```
a_vec %xn% (u_mat ~ 1)
#> [1] 40 46
```

We can also do a more complex example with a tensor and a matrix. Here, $\mathcal{A} \in \mathbb{R}^{2 \times 3 \times 4}$. Then if we compute $\mathcal{A} \times_2 \mathbf{U}$, we expect the final result to have dimensions $\in \mathbb{R}^{2 \times 2 \times 4}$.

```
A_tensor <- tensor(array(1:24, dim = c(2, 3, 4)))

dim(A_tensor %xn% (u_mat ~ 2))
#> [1] 2 2 4
```

In `tensortools`, n-mode products apply mode-specific covariance transformations using the Kronecker-separable covariance matrices. To generate tensor-normal observations, each mode of IID standard normal tensors are multiplied by corresponding Cholesky factors. For density evaluation and Mahalanobis-distance calculations, centered tensors are multiplied along each mode by the corresponding precision matrices.

Matricization

Matricization, also called unfolding, collapses a tensor into a matrix. It accepts an argument k that determines which mode to place in the rows, and then combines every other mode into the columns. This allows us to focus on variation along a specific mode.

Let $\mathcal{A} \in \mathbb{R}^{I_1 \times \ldots \times I_D}$ be an order D tensor. Its mode k matricization is a matrix $\mathbf{A}_{(k)} \in \mathbb{R}^{I_k \times \prod_{d \neq k} I_d}$, whose elements are taken column-wise from A. The mode k forms the rows of $\mathbf{A}_{(k)}$ while the remaining modes form the columns. Thus the entry $\mathcal{A}(i_1, \ldots, i_D)$ is in $\mathbf{A}_{(k)}(i_k, j)$ where j indexes the remaining modes.

In `tensortools`, this function is called `matricize()`. It accepts a tensor and k.

Example

The slices of $\mathcal{A}$ along the third mode are two 4×3 matrices.

$$A_{(:,:,1)} = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}, \quad A_{(:,:,2)} = \begin{bmatrix} 13 & 17 & 21 \\ 14 & 18 & 22 \\ 15 & 19 & 23 \\ 16 & 20 & 24 \end{bmatrix}.$$

Let us say we want to matricize along the first mode. This places the second and third mode into the columns of a matrix, so the resulting matrix belongs to $\mathbb{R}^{4 \times 6}$. We will do this by moving the values in the third mode into the columns.

The notation for matricization along the first mode is $A_{(1)}$.

$$A_{(1)} = \begin{bmatrix} 1 & 5 & 9 & 13 & 17 & 21 \\ 2 & 6 & 10 & 14 & 18 & 22 \\ 3 & 7 & 11 & 15 & 19 & 23 \\ 4 & 8 & 12 & 16 & 20 & 24 \end{bmatrix}$$

```r
A <- tensor(array(1:24, dim = c(4, 3, 2)))

matricize(A, k = 1)
#> <tensor[1]>
#> 1 draw of shape 4 × 6
#>
#> # Preview: 1 draw × 6 indexed entries
#>        [1,1] [2,1] [3,1] [4,1] [1,2] [2,2]
#> draw 1     1     2     3     4     5     6
#>
#> # ... 18 more entries per draw
```

In `tensortools`, matricization is used to initialize and update mode-specific covariance matrices through ordinary matrix crossproducts.

Tensor decompositions

The idea of decompositions can also apply to tensors. The package provides the Tucker, PARAFAC, higher-order singular value, and Tensor-Train decompositions.

The decompositions differ in how they represent low-dimensional structure and in how their ranks are specified:

Table 1: Types of Decompositions

Decomposition	Ranks	Core	Computation
Tucker	$(R_1, \dots, R_D)$	Dense	Iterative higher-order orthogonal iteration
CP/PARAFAC	R	Superdiagonal	Iterative alternating least squares
HOSVD	$(R_1, \dots, R_D)$	All-orthogonal	Modewise SVD
Tensor-Train	$(R_1, \dots, R_{D-1})$	Several cores	Sequential SVDs

Tucker decomposition

The idea behind the Tucker decomposition is similar to rank factorization for matrices. The Tucker decomposition factorizes an order-D tensor into a core tensor and a list of matrices (Hitchcock 1927):

$$\mathcal{A} = \mathcal{B} \times_1 \mathbf{U}_1 \times_2 \mathbf{U}_2 \cdots \times_D \mathbf{U}_D.$$

The core tensor $\mathcal{B}$ has the same order as the original tensor $\mathcal{A}$, and the number of $\mathbf{U}$ matrices is also the number of modes. The core $\mathcal{B}$ has dimensions $R_1 \times \cdots \times R_D$, and the tuple $(R_1, \ldots, R_D)$ is called the multilinear rank, or Tucker rank, of the approximation.

Each matrix $\mathbf{U}_n$ contains principal factors for mode n. The $\mathbf{U}_n$ is a basis for the nth mode, and thus spans a subspace of $\mathbb{R}^{I_n}$. The core tensor $\mathcal{B}$ represents the strength of interactions between factors. The idea behind this decomposition is to reduce the dimensionality of the input. We are able to summarize the higher-order object $\mathcal{A}$ with a smaller core tensor $\mathcal{B}$ and several factor matrices $\mathbf{U}_1, \mathbf{U}_2, \ldots, \mathbf{U}_D$. However, one key issue with the Tucker decomposition is that it is not unique.

The `tucker()` function uses higher-order orthogonal iteration initialized by the HOSVD (Kolda and Bader 2009). It alternately updates each factor matrix using the leading left singular vectors of the corresponding projected mode unfolding. Convergence is assessed using the relative Frobenius reconstruction error.

```
A <- array(1:24, dim = c(2, 3, 4))
res_tucker <- tucker(A, ranks = c(2, 2, 2))
glimpse(res_tucker)
#> List of 2
#>  $ G   : num [1:2, 1:2, 1:2] 69.88901 -0.01246 -0.00387 -0.21908 -0.00114 ...
#>  $ mats:List of 3
#>   ..$ : num [1:2, 1:2] -0.685 -0.728 -0.728 0.685
#>   ..$ : num [1:3, 1:2] 0.5029 0.5744 0.6459 0.7618 0.0584 ...
#>   ..$ : num [1:4, 1:2] -0.128 -0.337 -0.547 -0.756 -0.827 ...

A_tucker <- tucker_reconstruct(res_tucker)
frob_norm_diff(A_tucker, A)
#> [1] 5.1864e-16
```

This function does already exist in the `rTensor` package. However, it requires coercing any base R array into a tensor object. Additionally, its output computes the restructured tensor and its Frobenius norm. This behavior can be quite slow depending on how large the original tensor was.

```
rTensor_tucker <- rTensor::tucker(rTensor::as.tensor(A), ranks = c(2, 2, 2))
#>
  |
  |                                                                      |   0%
  |
```

```
  |===                                                                   |   4%
  |
  |======                                                                |   8%
  |
  |======================================================================| 100%
```

```
glimpse(rTensor_tucker)
#> List of 7
#>  $ Z           :Formal class 'Tensor' [package "rTensor"] with 3 slots
#>   .. ..@ num_modes: int 3
#>   .. ..@ modes    : int [1:3] 2 2 2
#>   .. ..@ data     : num [1:2, 1:2, 1:2] 69.88901 -0.01246 -0.00387 -0.21908 -0.00114 ...
#>  $ U           :List of 3
#>   ..$ : num [1:2, 1:2] -0.685 -0.728 -0.728 0.685
#>   ..$ : num [1:3, 1:2] 0.5029 0.5744 0.6459 0.7618 0.0584 ...
#>   ..$ : num [1:4, 1:2] -0.128 -0.337 -0.547 -0.756 -0.827 ...
#>  $ conv        : logi TRUE
#>  $ est         :Formal class 'Tensor' [package "rTensor"] with 3 slots
#>   .. ..@ num_modes: int 3
#>   .. ..@ modes    : int [1:3] 2 3 4
#>   .. ..@ data     : num [1:2, 1:3, 1:4] 1 2 3 4 5 ...
#>  $ norm_percent: num 100
#>  $ fnorm_resid : num 6.32e-14
#>  $ all_resids  : num [1:2] 3.62e-14 6.32e-14
```

PARAFAC decomposition

The canonical polyadic (CP) decomposition, also called CANDECOMP/PARAFAC, represents a tensor as a sum of rank-one tensors. PARAFAC stands for parallel factor analysis while CANDECOMP stands for canonical decomposition. This decomposition is a special case of the Tucker decomposition, where the core tensor is superdiagonal.

Let us say we want to approximate the tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_D}$ with a linear combination of R rank 1 tensors. The approximation can be written as

$$\mathcal{A} \approx \sum_{r=1}^{R} \lambda_r \mathbf{u}_1(:, r) \circ ... \circ \mathbf{u}_D(:, r).$$

where $\circ$ is a vector outer product.

However, determining the rank of a tensor is an NP-hard problem, so it is difficult to determine the value of R necessary to approximate the original tensor. The package computes this

decomposition using the alternating least squares algorithm described by Carroll and Chang (1970) and Harshman (1970). For each mode, `cp()` updates the corresponding factor matrix using the mode unfolding, the Khatri–Rao product of the remaining factor matrices, and their Gram matrices.

```
res_cp <- cp(A, R = 2)
glimpse(res_cp)
#> List of 2
#>  $ lambda: num [1:2] 88.2 27.1
#>  $ mats  :List of 3
#>   ..$ : num [1:2, 1:2] -0.694 -0.72 -0.731 -0.683
#>   ..$ : num [1:3, 1:2] -0.533 -0.576 -0.62 0.652 0.574 ...
#>   ..$ : num [1:4, 1:2] 0.324 0.432 0.539 0.647 0.742 ...

A_cp <- cp_reconstruct(res_cp)
frob_norm_diff(A_cp, A)
#> [1] 0.0009999903
```

This can also be done in `rTensor`.

```
rTensor_cp <- rTensor::cp(rTensor::as.tensor(A), num_components = 2)
#>
  |
  |                                                                      |   0%
  |
  |===                                                                   |   4%
  |
  |======                                                                |   8%
  |
  |========                                                              |  12%
  |
  |===========                                                           |  16%
  |
  |==============                                                        |  20%
  |
  |=================                                                     |  24%
  |
  |====================                                                  |  28%
  |
  |=======================                                               |  32%
  |
  |===========================                                           |  36%
```

```
  |
  |============================                                          |  40%
  |
  |================================                                      |  44%
  |
  |===================================                                   |  48%
  |
  |=======================================                               |  52%
  |
  |==========================================                            |  56%
  |
  |==============================================                        |  60%
  |
  |=================================================                     |  64%
  |
  |====================================================                  |  68%
  |
  |========================================================              |  72%
  |
  |===========================================================           |  76%
  |
  |===============================================================       |  80%
  |
  |==================================================================    |  84%
  |
  |======================================================================|  88%
  |
  |=========================================================================|  92%
  |
  |============================================================================|  96%
  |
  |================================================================================| 100%
```

```
glimpse(rTensor_cp)
#> List of 7
#>  $ lambdas     : num [1:2] 276 110
#>  $ U           :List of 3
#>   ..$ : num [1:2, 1:2] 0.478 0.522 -0.506 -0.494
#>   ..$ : num [1:3, 1:2] -0.274 -0.333 -0.393 -0.352 -0.333 ...
#>   ..$ : num [1:4, 1:2] -0.204 -0.235 -0.265 -0.296 -0.322 ...
#>  $ conv        : logi FALSE
#>  $ est         :Formal class 'Tensor' [package "rTensor"] with 3 slots
```

```
#>   .. ..@ num_modes: int 3
#>   .. ..@ modes    : int [1:3] 2 3 4
#>   .. ..@ data     : num [1:2, 1:3, 1:4] 1.08 1.91 3.01 3.99 4.95 ...
#>  $ norm_percent: num 99.5
#>  $ fnorm_resid : num 0.37
#>  $ all_resids  : num [1:24] 0.452 0.422 0.419 0.415 0.412 ...
```

Higher-order singular value decomposition

The higher-order singular value decomposition (HOSVD) is a specific case of the Tucker decomposition with restrictions. The matrices $\mathbf{U}$ have orthogonal columns and the core tensor $\mathcal{B}$ has subtensors, defined as setting the nth index to a value α, that are orthogonal (De Lathauwer et al. 2000). Additionally, the subtensors have a descending order of Frobenius norms:

$$\|\mathcal{B}_{i_n=1}\| \geq \|\mathcal{B}_{i_n=2}\| \geq ... \geq \|\mathcal{B}_{i_n=I_n}\| \geq 0.$$

These Frobenius norms are the n-mode singular values of $\mathcal{A}$, and the ith column $\mathbf{U}_n(:, i)$ is the ith n-mode singular vector.

For each mode d, `hosvd()` obtains $\mathbf{U}_d$ from the leading R_d left singular vectors of the mode-d matricization $\mathbf{A}_{(d)}$. The core is then obtained by multiplying $\mathcal{A}$ by the transposed factor matrices along their corresponding modes.

```
res_hosvd <- hosvd(A, ranks = c(2, 2, 2))
glimpse(res_hosvd)
#> List of 2
#>  $ G   : num [1:2, 1:2, 1:2] 69.88901 -0.01246 0.00387 0.21908 -0.00114 ...
#>  $ mats:List of 3
#>   ..$ : num [1:2, 1:2] -0.685 -0.728 -0.728 0.685
#>   ..$ : num [1:3, 1:2] -0.5029 -0.5744 -0.6459 0.7618 0.0584 ...
#>   ..$ : num [1:4, 1:2] 0.128 0.337 0.547 0.756 0.827 ...

A_hosvd <- hosvd_reconstruct(res_hosvd)
frob_norm_diff(A_hosvd, A)
#> [1] 1.725712e-15
```

This can also be done in `rTensor`.

```
rTensor_hosvd <- rTensor::hosvd(rTensor::as.tensor(A))
#>
  |
  |                                                                      |   0%
  |
  |=======================                                               |  33%
  |
  |===============================================                       |  67%
  |
  |======================================================================| 100%

glimpse(rTensor_hosvd)
#> List of 4
#>  $ Z          :Formal class 'Tensor' [package "rTensor"] with 3 slots
#>   .. ..@ num_modes: int 3
#>   .. ..@ modes    : int [1:3] 2 3 4
#>   .. ..@ data     : num [1:2, 1:3, 1:4] 6.99e+01 -1.25e-02 3.87e-03 2.19e-01 -6.79e-15 ..
#>  $ U          :List of 3
#>   ..$ : num [1:2, 1:2] -0.685 -0.728 -0.728 0.685
#>   ..$ : num [1:3, 1:3] -0.5029 -0.5744 -0.6459 0.7618 0.0584 ...
#>   ..$ : num [1:4, 1:4] 0.128 0.337 0.547 0.756 0.827 ...
#>  $ est        :Formal class 'Tensor' [package "rTensor"] with 3 slots
#>   .. ..@ num_modes: int 3
#>   .. ..@ modes    : int [1:3] 2 3 4
#>   .. ..@ data     : num [1:2, 1:3, 1:4] 1 2 3 4 5 ...
#>  $ fnorm_resid: num 1.19e-13
```

Tensor-Train decomposition

The Tensor-Train decomposition factorizes an order-D tensor $\mathcal{A}$ in a product of third-order tensors (Oseledets 2011). These are denoted as $\mathbf{U}_1, \mathcal{U}_2, \ldots, \mathbf{U}_D$ and are called TT-cores:

$$\mathcal{A}(i_1, \ldots, i_D) = \mathbf{U}_1(i_1, :)\mathcal{U}_2(:, i_2, :)\mathcal{U}_3(:, i_3, :) \ldots \mathbf{U}_D(:, i_D).$$

The package computes this decomposition using the TT-SVD algorithm. The user can provide the ranks directly or allow them to be selected using an approximation accuracy. When `epsilon` is supplied, `tt()` uses the TT-SVD truncation threshold

$$\delta = \frac{\epsilon}{\sqrt{D-1}} \|\mathcal{A}\|_F.$$

The resulting approximation $\widehat{\mathcal{A}}$ satisfies

$$\left\|\mathcal{A} - \widehat{\mathcal{A}}\right\|_F \leq \epsilon \left\|\mathcal{A}\right\|_F .$$

```
res_tt <- tt(A, ranks = c(2, 2, 2))
glimpse(res_tt)
#> List of 3
#>  $ : num [1:2, 1:2] -0.685 -0.728 -0.728 0.685
#>  $ : num [1:2, 1:3, 1:2] -0.502874 0.002478 -0.574387 0.000285 -0.645901 ...
#>  $ : num [1:2, 1:4] 8.97 -3.25 23.58 -1.7 38.2 ...

A_tt <- tt_reconstruct(res_tt)
frob_norm_diff(A_tt, A)
#> [1] 8.967358e-16

res_tt_epsilon <- tt(A, epsilon = 1e-2)
A_tt_epsilon <- tt_reconstruct(res_tt_epsilon)
frob_norm_diff(A_tt_epsilon, A)
#> [1] 8.967358e-16
```

This can be done in the package **ttTensor**. In this package, the input tensor must specifically have dimension names. It also has **rTensor** as a dependency.

```
A_ttTensor <- A

dimnames(A_ttTensor) <- list(I = paste0("i", 1:2),
                             J = paste0("j", 1:3),
                             K = paste0("k", 1:4))

tt_ranks <- ttTensor::TTSVD(rTensor::as.tensor(A_ttTensor), Ranks = c(p=2, q=2, r=2))
tt_epsilon <- ttTensor::TTSVD(rTensor::as.tensor(A_ttTensor), accuracy = 1e-2)

glimpse(tt_ranks)
#> List of 3
#>  $ I-p  : num [1:2, 1:2] -0.685 -0.728 -0.728 0.685
#>   ..- attr(*, "dimnames")=List of 2
#>   .. ..$ I: chr [1:2] "i1" "i2"
#>   .. ..$ p: chr [1:2] "p1" "p2"
#>  $ p-J-q: num [1:2, 1:3, 1:2] -0.502874 0.002478 -0.574387 0.000285 -0.645901 ...
#>   ..- attr(*, "dimnames")=List of 3
#>   .. ..$ p: chr [1:2] "p1" "p2"
#>   .. ..$ J: chr [1:3] "j1" "j2" "j3"
```

```
#>   .. ..$ q: chr [1:2] "q1" "q2"
#>  $ q-K  : num [1:2, 1:4] 8.97 -3.25 23.58 -1.7 38.2 ...
#>   ..- attr(*, "dimnames")=List of 2
#>   .. ..$ q: chr [1:2] "q1" "q2"
#>   .. ..$ K: chr [1:4] "k1" "k2" "k3" "k4"
```

Tensor-variate distributions

This section introduces the distributions that are used by `tensortools`, which are generalizations of matrix variate distributions.

Univariate normal

First, consider a standard univariate normal draw $Z \sim N(0,1)$. If we shift and scale it to generate $X = \mu + \sigma Z$, then $X \sim N(\mu, \sigma^2)$. That is, an affine transformation of a standard univariate normal draw has a univariate normal distribution with mean μ and variance σ^2.

Multivariate normal

Now let $\mathbf{Z}$ be a $p \times 1$ vector containing IID standard normal draws. Then $\mathbf{Z} \sim N_p(\mathbf{0}, \mathbf{I}_p)$. If we let $\mathbf{A}\mathbf{A}^T = \mathbf{\Sigma}$, then define $\mathbf{X} = \boldsymbol{\mu} + \mathbf{A}\mathbf{Z}$. Now $\mathbf{X} \sim N_p(\boldsymbol{\mu}, \mathbf{\Sigma})$. That is, $\mathbf{X}$ has a multivariate normal distribution with mean $\boldsymbol{\mu}$ and covariance $\mathbf{\Sigma}$.

Matrix-variate normal

Let $\mathbf{Z}$ be a $p \times q$ matrix with IID standard normal entries. Choose matrices $\mathbf{A}$ and $\mathbf{B}$ such that $\mathbf{U} = \mathbf{A}\mathbf{A}^T, \mathbf{V} = \mathbf{B}\mathbf{B}^T$. Now define $\mathbf{X} = \mathbf{M} + \mathbf{A}\mathbf{Z}\mathbf{B}^T$. We say that $\mathbf{X} \sim N_{pq}(\mathbf{M}, \mathbf{U}, \mathbf{V})$. That is, $\mathbf{X}$ has a matrix variate normal distribution.

Tensor-variate normal

Let $\mathcal{Z}$ be an order-D tensor with IID standard normal entries and $\mathbf{\Sigma}_1, \ldots, \mathbf{\Sigma}_D$ be positive definite matrices describing the variance-covariance structure along each mode. For each mode, choose $\mathbf{A}_j$ such that $\mathbf{\Sigma}_d = \mathbf{A}_d \mathbf{A}_d^T, d = 1, \ldots, D$.

Then we can define $\mathcal{X}$ as $\mathcal{X} = \mathcal{M} + \mathcal{Z} \times_1 \mathbf{A}_1 \times_2 \mathbf{A}_2 \cdots \times_D \mathbf{A}_D$. That is, $\mathcal{X}$ has a tensor variate normal distribution with mean $\mathcal{M}$ and covariance structure described by $\bigotimes_{d=D}^{1} \Sigma_d$. The notation $\bigotimes_{d=D}^{1} \Sigma_d$ is shorthand for $\Sigma_D \otimes \cdots \otimes \Sigma_1$.

Table 2: Mixing distributions used to construct the skewed tensor variate distributions.

Distribution	Distribution of W	Parameters	Parameter support
Skewed-t	Inv-Gamma$(\nu/2, \nu/2)$	ν	$\nu > 0$
Generalized hyperbolic	Gen Inv Gaussian$(\omega, 1, \lambda)$	λ, ω	$\omega > 0,\ \lambda \in \mathbb{R}$
Variance gamma	Gamma(γ, γ)	γ	$\gamma > 0$
Normal inverse Gaussian	Inv Gaussian$(1, \kappa)$	κ	$\kappa > 0$

Skewed distributions as transformations of the tensor variate normal

To generate tensor variate skewed distributions, we use the normal variance mean mixture model (Barndorff-Nielsen et al. 1982). An r-dimensional random vector x is a normal variance-mean mixture with mixing distribution F if, for a given $u \geq 0$ that follows a probability distribution F on $[0, \infty)$, $x|u \sim N_r(\mu + u\beta, u\Sigma)$. $\Sigma \in \mathbb{R}^{r \times r}$ is a constant, positive-definite matrix and $\mu \in \mathbb{R}^r$ and $\beta \in \mathbb{R}^r$ are constant vectors.

We state that x is a normal variance-mean mixture with position μ, drift β, structure matrix Σ, and mixing distribution F. Different mixing distributions lead to different distributions. For example, if F, the mixing distribution, is the generalized inverse Gaussian distribution, then the resulting distribution of x is a generalized hyperbolic distribution.

In the tensor variate case, the normal variance mean mixture model is an efficient way to introduce skewness. A random tensor $\mathcal{X}$ can be written as

$$\mathcal{X} = \mathcal{M} + W\mathcal{A} + \sqrt{W}\mathcal{V}, \tag{1}$$

where $\mathcal{M}$ is a location tensor, $\mathcal{A}$ is a skewness tensor, $\text{vec}(\mathcal{V}) \sim \mathcal{N}_{I^*}(\mathbb{0}, \bigotimes_{d=D}^{1} \mathbf{\Sigma}_d)$ where $I^* = \prod_{d=1}^{D} I_d$ and $W > 0$ is a positive random variable. The distribution of $\mathcal{X}$ will depend on the distribution of W.

The mixing distributions and their parameters for each of the skewed tensor variate distributions are summarized below. In each case, the support of W is $(0, \infty)$.

These choices of W give the four skewed families implemented in `tensortools`. Their common representation also makes it possible to use a shared estimation framework while changing the distribution of the latent mixing variable.

Tensor software across programming languages

There are several existing software packages for tensors across several programming languages, but most emphasize tensor algebra or decompositions over probability models. The distinguishing feature of `tensortools` is the support for tensor-variate normal and skewed tensor-variate distributions. These include random number generation, density functions, likelihood estimation, and model diagnostics.

One of the most popular software packages is the `Tensor Toolbox` for MATLAB (Bader et al. 2025; Bader and Kolda 2006). This toolbox introduces a `tensor` class to generate dense tensors from a multidimensional MATLAB array. This class object has unique attributes so that the user can use methods such as `size`, `ndims`, `norm`, `unfold`, and `ttm`. It also supports decompositions on the tensor object with methods like `mttkrp`, `cp_als`, `tucker_als`, and `hosvd`. `Tensor Toolbox` also defines separate classes for other tensor formats: `sptensor` for sparse tensors, `ktensor` for Kruskal/CP factored tensors, and `ttensor` for Tucker factored tensors (Bader and Kolda 2007).

`TensorLy` in Python is similar to the Tensor Toolbox for MATLAB. It contains basic tensor operations and common decompositions including CP, Tucker, and Tensor-Train. A major strength of `TensorLy` is that it has backend support across other common frameworks in Python, including `NumPy`, `PyTorch`, and `TensorFlow`. This makes it easier for the user to run functions on their GPU or along multiple machines (Kossaifi et al. 2019). `TensorFlow` is another well-known package in Python, but it primarily uses the tensor object as inputs for machine learning methodologies rather than trying to expand on operations and decompositions.

The Python Tensor Toolbox, `pyttb` is a refactoring of the Tensor Toolbox for MATLAB. Thus, it ports the common functions from Tensor Toolbox into Python, including the various unique tensor classes and decompositions (pyttb developers 2026).

In Julia, `TensorOperations.jl` provides support for tensor contractions and `TensorToolbox.jl` is Julia's port of the MATLAB Tensor Toolbox (Periša 2023; TensorOperations.jl developers 2026).

In R, `rTensor` has an S4 `Tensor` class and includes common operations and decompositions. It contains the CP, Tucker, and HOSVD (Li, Bien, and Wells 2024). `ttTensor` adds the Tensor-Train decomposition to this framework (Li, Bien, Wells, Xiao, et al. 2024). `tensorr` is now archived on CRAN, but it originally provided support for S4 sparse tensor and dense tensor classes.

Table 3: Comparison of tensor operations, decompositions, and statistical functionality across software packages.

Package	R package	Tensor operations	Decompositions	Tensor normal	Skewed families	Covariance LRT	Goodness of fit
tensortools	✓	✓	✓	✓	✓	✓	✓
tensr	✓	✓	✓	✓	×	✓	×
rTensor	✓	✓	✓	×	×	×	×
ttTensor	✓	×	✓	×	×	×	×
MATLAB Tensor Toolbox	×	✓	✓	×	×	×	×
TensorLy	×	✓	✓	×	×	×	×
pyttb	×	✓	✓	×	×	×	×

Package design

The main goal of `tensortools` is to provide functions generating tensor observations, evaluating densities, estimating parameters using the maximum likelihood estimators, and diagnostic tests to assess model fit. Specifically, it supports the tensor-variate normal distribution and four normal variance-mean mixture families: skewed-t, generalized hyperbolic, variance gamma, and normal inverse Gaussian.

Distribution functions follow familiar R naming conventions: names beginning with `r` generate random draws, names beginning with `d` evaluate densities, and `tensor_mle()` fits a selected tensor-variate model.

Tensor algebra helpers such as `matricize()`, `n_prod()`, `nm_prod()`, `tensor_prod()`, and `khatri_rao()` are available to users and are also used internally by the likelihood and decomposition routines. Decomposition functions are grouped with explicit reconstruction helpers: `cp()` and `cp_reconstruct()`, `tucker()` and `tucker_reconstruct()`, `hosvd()` and `hosvd_reconstruct()`, and `tt()` and `tt_reconstruct()`.

One intentional decision was to store independent, identically distributed draws from a tensor variate distribution as a lightweight S3 array subclass. For example, if we generate $N = 100$ draws from a tensor variate normal distribution of dimension $2 \times 3 \times 4$, the resulting R object will be a tensor class.

There are unique class functions to emphasize that there are N independent, identically distributed draws from the same distribution. `n_draws()` returns the number of draws while `draw_shape()` outputs the dimensions of one draw.

```
norm_three <- rtnorm(n = 100, mu = array(1:24, dim = c(2, 3, 4)))
n_draws(norm_three)
#> [1] 100
draw_shape(norm_three)
#> [1] 2 3 4
```

Additionally, `pull_draw()` lets a user pull a specific draw by its index. The native `R` subsetting is still supported using `[`, but it can be unwieldy to type the commas required to subset across every other dimension.

```
pull_draw(norm_three, 1)
#> <tensor[1]>
#> 1 draw of shape 2 × 3 × 4
#>
#> # Preview: 1 draw × 6 indexed entries
#>          [1,1,1]  [2,1,1]  [1,2,1]  [2,2,1]  [1,3,1]  [2,3,1]
#> draw 1 0.1952812 4.288733 4.065558 4.737389 5.421633 5.591133
#>
#> # ... 18 more entries per draw
norm_three[1, , ]
#> <tensor[1]>
#> 1 draw of shape 2 × 3 × 4
#>
#> # Preview: 1 draw × 6 indexed entries
#>          [1,1,1]  [2,1,1]  [1,2,1]  [2,2,1]  [1,3,1]  [2,3,1]
#> draw 1 0.1952812 4.288733 4.065558 4.737389 5.421633 5.591133
#>
#> # ... 18 more entries per draw
```

Model fits are returned as ordinary R lists. For example, a tensor-variate normal fit contains the estimated mean array `mu`, the mode-specific covariance matrices `sigmas`, likelihood-based summaries, and convergence information when relevant. Skewed families add distribution-specific parameters and skewness arrays. This return design keeps the model output easy to inspect with base R tools while avoiding a separate hierarchy of model classes.

Basic package usage

This package contains random-number generation for several tensor variate distributions. Below is a simulation from a tensor variate normal of size $2 \times 3 \times 4$. The shape of each draw is the same as the mu object provided. The covariance structure is specified with a list of matrices, where each matrix must be a square matrix whose dimension matches the corresponding mode.

```
mu_true <- tensor(array(1:24, dim = c(2, 3, 4)))

S1 <- crossprod(matrix(rnorm(4), nrow = 2))
S2 <- crossprod(matrix(rnorm(9), nrow = 3))
S3 <- crossprod(matrix(rnorm(16), nrow = 4))
```

```r
sigmas_true <- list(S1, S2, S3)

tvn_draws <- rtnorm(n = 1e3, mu = mu_true, sigmas = sigmas_true)
```

Given a `tensor` object, the package can estimate the MLEs using the function `tensor_mle()`. The model argument is required to know which distribution to fit to the data.

```r
tvn_mles <- tensor_mle(tvn_draws, model = "normal")

str(tvn_mles)
#> List of 6
#>  $ mu    : num [1:2, 1:3, 1:4] 0.96 2 3.01 4.01 4.99 ...
#>  $ sigmas:List of 3
#>   ..$ : num [1:2, 1:2] 1.536 0.441 0.441 0.464
#>   ..$ : num [1:3, 1:3] 0.722 -0.482 -0.064 -0.482 1.213 ...
#>   ..$ : num [1:4, 1:4] 0.3979 -0.09194 0.10264 -0.00727 -0.09194 ...
#>  $ loglik: num -10238
#>  $ k     : num 41
#>  $ AIC   : num 20557
#>  $ BIC   : num 20758
```

There are also density functions for each distribution. For the tensor variate normal, the required arguments are mu and sigmas. There is an optional log argument to compute the log density rather than the density itself.

```r
dtnorm(pull_draw(tvn_draws, 1), mu = mu_true, sigmas = sigmas_true)
#> [1] 4.896922e-06
```

Model fitting and inference

One major functionality that tensortools provides is MLE estimation. This is done via two different algorithms: for the tensor variate normal distribution, a flip-flop algorithm is used. For tensor variate skewed distributions, an EM algorithm is used.

The separable covariance restriction and its curved exponential-family representation are described in the Appendix section on the tensor variate normal model.

The initialization rules for the flip-flop and EM algorithms are provided in the Appendix section on parameter initialization.

The individual covariance matrices in a Kronecker product are not defined uniquely. Multiplying one covariance matrix by a constant and another by the inverse of that constant produces the same full covariance matrix. The package addresses this by normalizing the trace of each covariance matrix except the final one to equal its corresponding mode dimension. This preserves the overall Kronecker covariance while giving the individual mode covariance estimates a consistent scale.

The flip-flop update equations and convergence criterion are provided in the Appendix sections on flip-flop estimation and convergence.

Diagnostic Tests

Rosenblatt Interpoint Goodness of Fit Test

We now discuss a diagnostic test that combines the Rosenblatt transformation, the interpoint-distance uniformity test, and an MLE adjustment for U-statistics. This test applies to the tensor variate normal and normal variance-mean mixture family distributions.

First, we define the Rosenblatt transformation. The Rosenblatt transformation is the multivariate extension of the probability integral transformation Rosenblatt (1952). It applies the conditional CDF of each coordinate given the coordinates before it. This removes the modeled dependence sequentially. If the proposed joint distribution is correct, the resulting transformed data are independent Uniform(0, 1) random variables.

Let

$$\mathbf{X} = (X_1, \dots, X_p)^T$$

be a continuous random vector with joint distribution function $F(x_1, \dots, x_p)$.

We define the Rosenblatt transformation as follows:

$$\begin{aligned}
U_1 &= F_{X_1}(X_1), \\
U_2 &= F_{X_2|X_1}(X_2 \mid X_1), \\
U_3 &= F_{X_3|X_1,X_2}(X_3 \mid X_1, X_2), \\
&\vdots \\
U_p &= F_{X_p|X_1,\dots,X_{p-1}}(X_p \mid X_1, \dots, X_{p-1}).
\end{aligned}$$

If the assumed joint distribution is correct and its parameters are known, then

$$\mathbf{U} = (U_1, \dots, U_p)^T \sim \text{Uniform}([0,1]^p).$$

Each coordinate is transformed using its distribution conditional on all the coordinates that came before it. The function `tensor_rosenblatt()` orders the coordinates according to `R`'s column-major vectorization convention. Under the alternative, the transformation is order-dependent.

The plot below shows the sequential transformation. First, X_1 is transformed into U_1. After that, X_2 is transformed using its conditional distribution given X_1. Under the correct model, the resulting points (U_1, U_2) are uniformly distributed in $[0, 1]^2$.

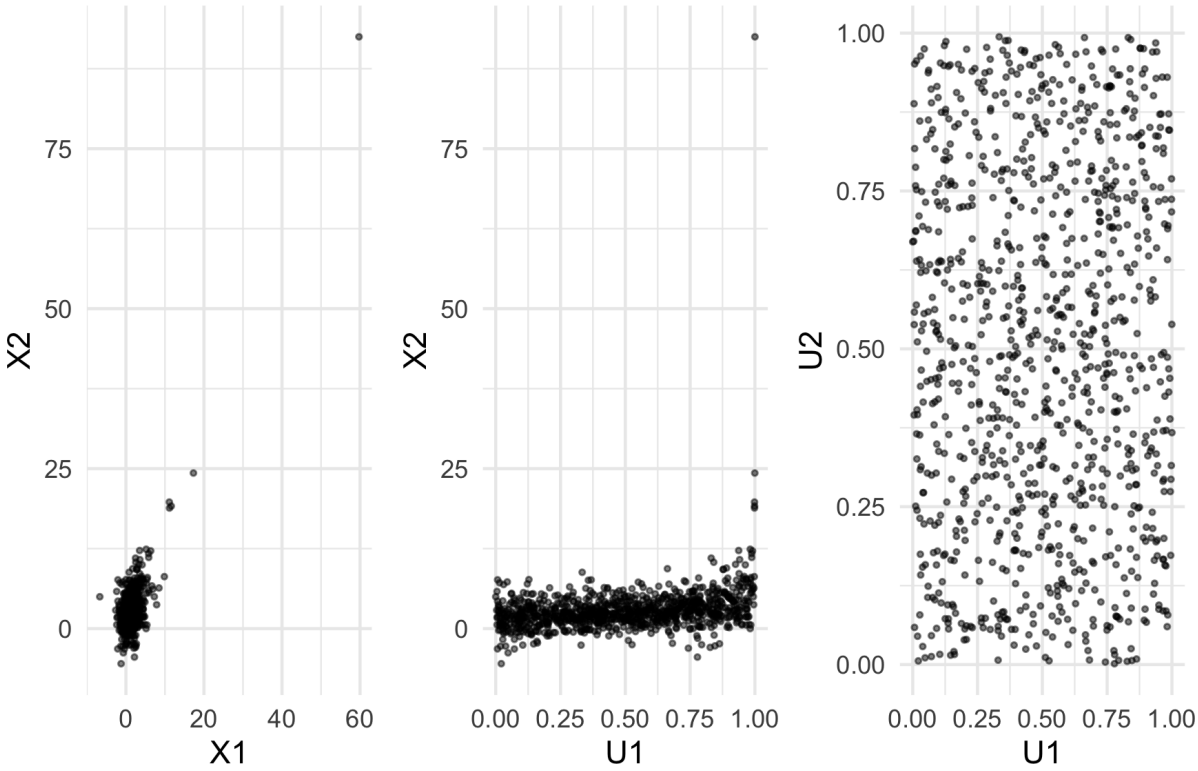


Figure 2: The plots from left to right show the Rosenblatt transformation being applied sequentially. The leftmost plot contains the points of the original data, the middle has X_1 transformed into U_1, and the right has X_2 transformed into U_2.

Now that we have our uniform draws, we need a formal diagnostic test to assess them. Following Yang and Modarres (2017), we apply the interpoint-distance test for $p \geq 2$ where $p = \prod_{d=1}^{D} I_d$ is the dimension of the vectorized tensor observation.

First, they define the interpoint distance of two points as

$$D_{ij} = ||\mathbf{U}_i - \mathbf{U}_j||^2$$

We can define the sample mean of the interpoint distances as

$$\bar{\delta} = \frac{1}{\binom{N}{2}} \sum_{1 \le i < j \le N}^{N} D_{ij}$$

The second moment of the interpoint distances centered around the null mean $p/6$ is

$$\bar{S} = \frac{1}{\binom{N}{2}} \sum_{1 \le i < j \le N}^{N} \left(D_{ij} - \frac{p}{6} \right)^2$$

The null variances, known-parameter test statistics, and their reference distributions are given in the Appendix section on the known-parameter null distribution.

Now we have two formal test statistics. However, these reference distributions for Q_1 and Q_2 assume that the parameters used in the Rosenblatt transformation are known. In practice, `tensor_mle()` estimates the parameters as MLEs from the same observations. We therefore need to adjust the asymptotic variance of $\bar{\delta}$ and $\bar{S}$ to obtain accurate test statistics.

The derivation of the fitted-parameter adjustment is provided in the Appendix section on the adjustment for estimated parameters.

The adjusted statistics are

$$Q_{1,\text{adj}} = \frac{N(\bar{\delta} - p/6)^2}{\hat{\Omega}_{1,\text{adj}}} \xrightarrow{D} \chi^2_1,$$
$$Q_{2,\text{adj}} = \frac{N(\bar{S} - 7p/180)^2}{\hat{\Omega}_{2,\text{adj}}} \xrightarrow{D} \chi^2_1$$

The adjusted Q_1 statistic can be computed using the `adjusted_q1()` function. It returns the test statistic and p-value.

```
bivar_mle <- tensor_mle(bivar_skewt, model = "skewt")

adjusted_q1(bivar_skewt, theta_hat = bivar_mle,
            rosenblatt_fun = rosenblatt_fun)
#> $U1
#> [1] -0.00414208
#>
#> $variance_adjusted
#> [1] 0.002990185
#>
#> $statistic
```

```
#> [1] 5.737713
#>
#> $p.value
#> [1] 0.0166044
```

Mahalanobis Distance

We assess tensor variate normality using the Mahalanobis distance. If $\mathcal{X}$ follows an order-D tensor variate normal distribution, then $\text{vec}(\mathcal{X})$ follows a multivariate normal distribution with mean $\text{vec}(\mathcal{M})$ and covariance matrix $\mathbf{\Sigma}_D \otimes \cdots \otimes \mathbf{\Sigma}_1$. The tensor distance uses the fitted separable covariance, while the vector distance uses an unrestricted covariance estimate.

When the separable covariance structure is appropriate, the multivariate and tensor variate distances should be similar. This can be visualized with a distance-distance plot: when the tensor normal structure is appropriate, the distances lie close to the reference line $y = x$; when the Kronecker product structure is absent, the distances diverge.

The function `mahalanobis_dist()` computes the tensor Mahalanobis distances using fitted MLEs, while `mahalanobis_compare()` computes both the tensor and vectorized distances. The function `mahalanobis_test()` performs a two-sample Kolmogorov–Smirnov test to compare their empirical distributions.

The figure below shows the Mahalanobis distance-distance plots along with the KS test statistic and p-value.

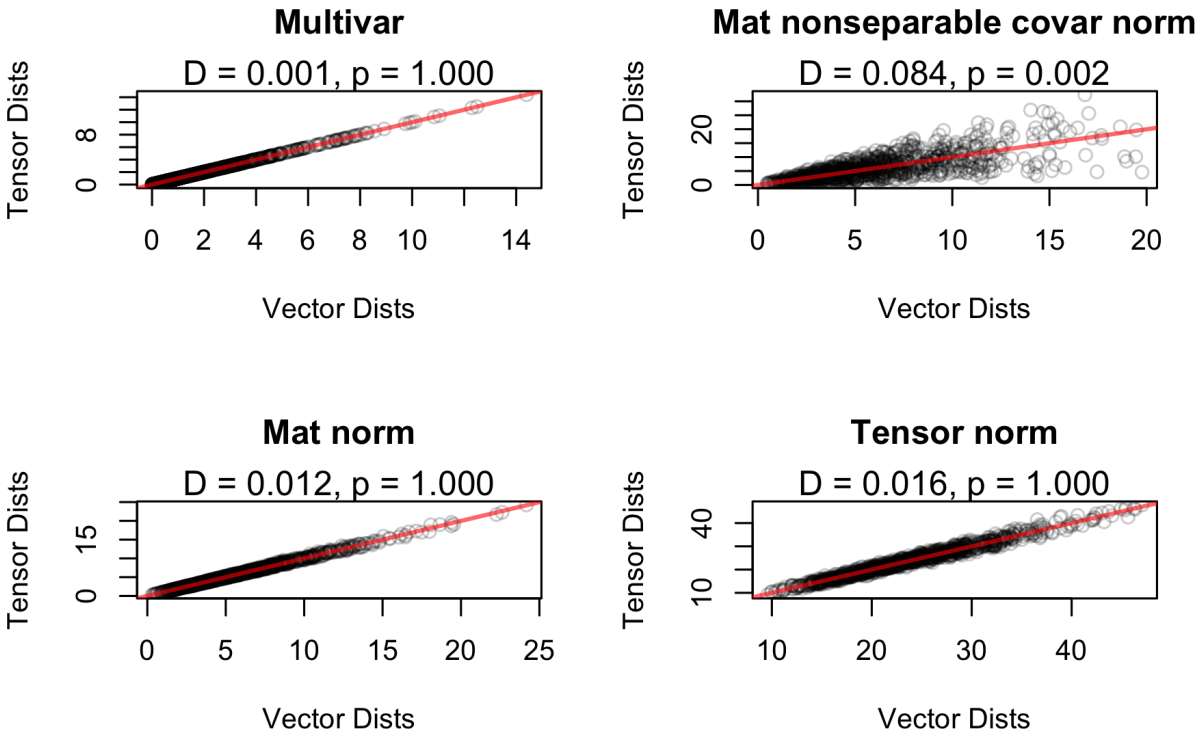


Figure 3: From left to right, the input data is a multivariate normal, matrix variate normal with a separable covariance structure, matrix variate with nonseparable covariance matrices, and tensor variate normal with a separable covariance structure. When the separable covariance structure is appropriate, the points should lie near the reference line $y = x$. There should be a small KS statistic D value and a large p-value.

Likelihood Ratio Tests

We are also interested in performing a likelihood ratio test to see if one of the covariance matrices might be the identity matrix. Thus, we run the algorithm and restrict these matrices.

For a restriction set $\mathcal{R} \subset \{1, \ldots, D\}$, the restricted model fixes $\Sigma_r = I_{I_r}$ for every $r \in \mathcal{R}$, while all other covariance matrices are estimated. The likelihood ratio test compares

$$H_0 : \Sigma_r = I_{I_r} \ \forall r \in \mathcal{R} \quad \text{vs.} \quad H_A : \Sigma_r \neq I_{I_r} \text{ for at least one } r \in \mathcal{R}.$$

The test statistic is

$$2\{\ell(\hat{\theta}_{\text{full}}) - \ell(\hat{\theta}_{\mathcal{R}})\} \sim \chi^2_{\nu_{\mathcal{R}}}, \quad \nu_{\mathcal{R}} = \sum_{r \in \mathcal{R}} \left\{ \frac{I_r(I_r+1)}{2} - 1 \right\} + 1(D \in \mathcal{R})$$

If the p-value is small, we reject the null hypothesis, which states the unrestricted model has a significantly better fit than the restricted model.

```
#> # A tibble: 1 x 4
#>   restrict statistic    df `p-val`
#>   <chr>        <dbl> <dbl> <chr>
#> 1 1            7879.     2 <1e-16
```

As expected, all of the restrictions lead to a model that is significantly different from the full model. All the covariance matrices are not the identity to best model the draws.

```
res <- map_dfr(list(1, 2, 3, c(1, 2), c(1, 3), c(2, 3)),
               draws = tvn_draws, test_restrict)
```

Table 4: LRT for Full Tensor Variate

restrict	statistic	df	p-val
1	7878.678	2	<1e-16
2	7934.139	5	<1e-16
3	33245.209	9	<1e-16
1, 2	15737.452	7	<1e-16
1, 3	41275.732	11	<1e-16
2, 3	41548.484	14	<1e-16

Now, we generate data where the second covariance matrix is the identity. The model that restricts the 2nd covariance to be identity has a $p - val > 0.05$, meaning it does not have a significantly smaller likelihood than the full unrestricted model.

Table 5: LRT for Second Sigma Identity

restrict	statistic	df	p-val
1	12266.820	2	<1e-16
2	10.163	5	0.0708
3	33693.949	9	<1e-16
1, 2	12281.010	7	<1e-16
1, 3	45645.457	11	<1e-16
2, 3	33711.996	14	<1e-16

Performance and validation

Parameter recovery

To validate the distributions, we can compare the MLEs to the true values used to generate draws.

```
mean((tvn_mles$mu - mu_true)^2)
#> [1] 0.0005690286

for(i in 1:3) {
  true_scaled <- sigmas_true[[i]] / sum(diag(sigmas_true[[i]]))
  est_scaled <- tvn_mles$sigmas[[i]] / sum(diag(tvn_mles$sigmas[[i]]))

  print(mean((est_scaled - true_scaled)^2))
}
#> [1] 3.177232e-06
#> [1] 1.650695e-05
#> [1] 5.290741e-06

true_cov <- Reduce(kronecker, rev(sigmas_true))
est_cov  <- Reduce(kronecker, rev(tvn_mles$sigmas))

sqrt(sum((est_cov - true_cov)^2))/sqrt(sum(true_cov^2))
#> [1] 0.02261964
```

Example

A common place where tensors arise is in the learning parameters of neural networks. Consider a dataset of ResNet-18 models trained on CIFAR10. ResNet-18 is an 18-layer convolutional neural network and CIFAR-10 is a dataset containing 32x32 color images Schürholt et al. (2022). We can download the parameters used in these neural networks and run a tensor variate normal model on them.

As seen in the table below, the tensor variate model had the smallest BIC. It had a comparable likelihood to the matricizations while having far fewer parameters.

```
load("obs_resnet.Rdata")

fits <- list(
  tensor = tensor_mle(obs, model = "normal", tol = 1e-6),
```

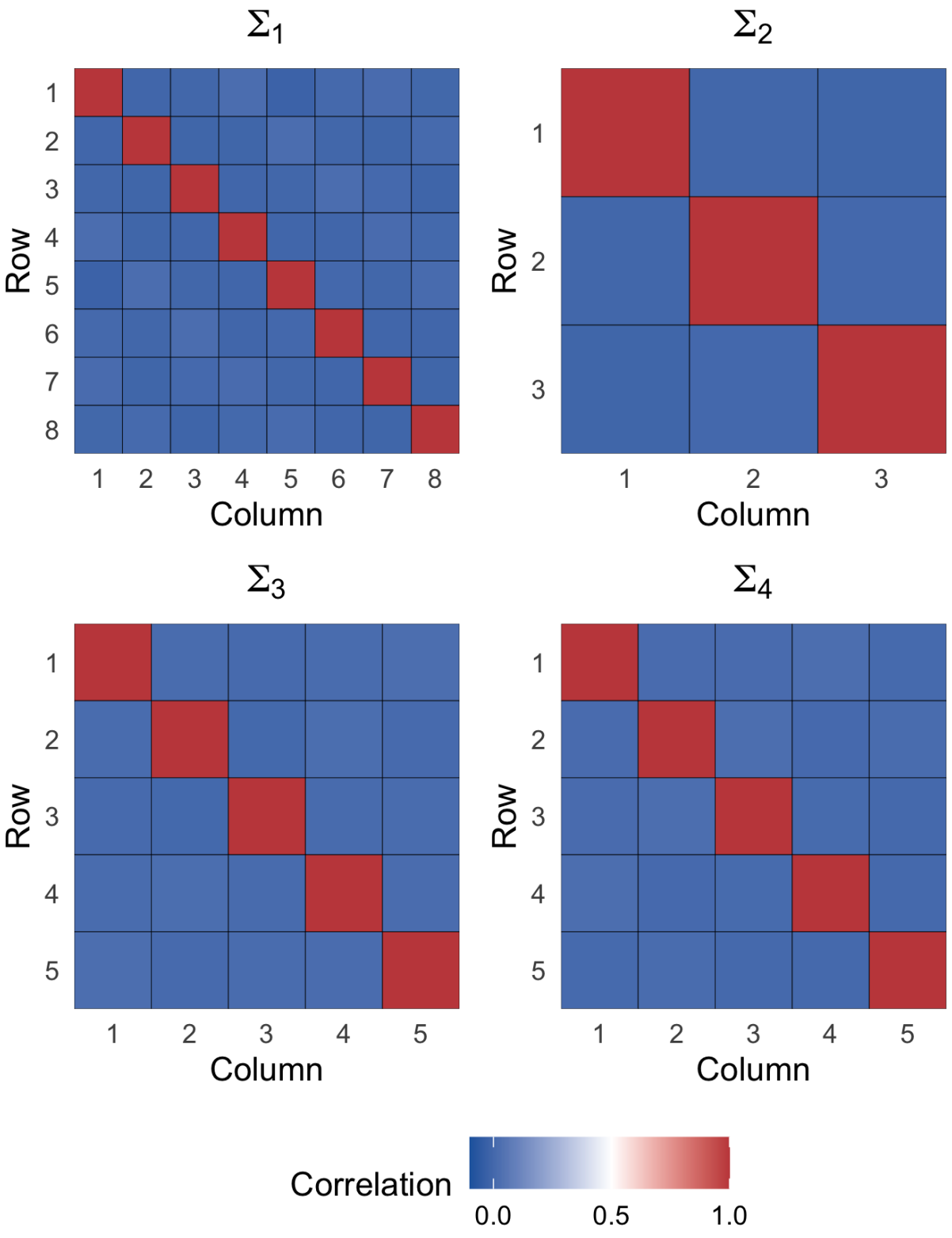
```
  first_mat = tensor_mle(matricize(obs, 1), model = "normal", tol = 1e-6),
  second_mat = tensor_mle(matricize(obs, 2), model = "normal", tol = 1e-6),
  third_mat = tensor_mle(matricize(obs, 3), model = "normal",  tol = 1e-6))

resnet_res <-
  tibble(model = names(fits),
         loglik = map_dbl(fits, "loglik"),
         BIC = map_dbl(fits, "BIC"),
         k = map_dbl(fits, "k")) |>
  arrange(BIC)

knitr::kable(resnet_res, escape = TRUE, booktabs = TRUE)
```

model	loglik	BIC	k
tensor	480100.8	-955580.2	669
first_mat	485754.4	-947435.2	3485
third_mat	492381.3	-930370.8	7874
second_mat	514395.4	-885765.8	20705

Below are the MLEs for the correlation matrices under the tensor variate normal model.


Σ_1
Σ_2
Σ_3
Σ_4
Row
Column
1 2 3 4 5 6 7 8
1 2 3
1 2 3 4 5
Correlation
0.0 0.5 1.0

Discussion

The package `tensortools` provides support for tensor variate distributions, allowing for random number generation, density calculation, and MLE estimation.

Reproducibility

```
#> R version: 4.5.2
#> Platform: aarch64-apple-darwin20
#> rjtools version: 1.0.21
```

Acknowledgments

Add funding, contributor, and institutional acknowledgments here.

Appendix

Tensor Variate Normal as a Curved Exponential Family

Setting this restriction leads to a curved exponential family. To see this, first consider the ordinary multivariate normal distribution. If $y \sim (\mu, \Sigma) \in \mathbb{R}^{np}$, then

$$f(y; \mu, \Sigma) = (2\pi)^{-np/2}|\Sigma|^{-1/2} \exp\left\{-\frac{1}{2}(y-\mu)^T\Sigma^{-1}(y-\mu)\right\}.$$

In exponential family form, the natural parameters are functions of $\Sigma^{-1}\mu$ and Σ^{-1}. When Σ is unrestricted, Σ^{-1} can be any positive definite $np \times np$ matrix.

Now let $\mathbf{X} \sim N(\mu, \mathbf{U}, \mathbf{V}) \in \mathbb{R}^{n\times p}$ and set $y = \text{vec}(\mathbf{X})$. The matrix variate normal distribution imposes the separable covariance restriction

$\Sigma = \mathbf{V} \otimes \mathbf{U}$

where $\mathbf{U}$ is the row covariance matrix and $\mathbf{V}$ is the column covariance matrix. Therefore $\text{vec}(\mathbf{X}) \sim N_{np}\left(\text{vec}(\mathbf{M}), \mathbf{V} \otimes \mathbf{U}\right)$

and the density becomes

$$f(\mathbf{X}; \mathbf{M}, \mathbf{U}, \mathbf{V}) = (2\pi)^{-np/2}|\mathbf{V} \otimes \mathbf{U}|^{-1/2} \exp\left\{-\frac{1}{2}\text{vec}(\mathbf{X}-\mathbf{M})^T(\mathbf{V}^{-1} \otimes \mathbf{U}^{-1})\text{vec}(\mathbf{X}-\mathbf{M})\right\}.$$

Equivalently, using $|\mathbf{V} \otimes \mathbf{U}| = |\mathbf{V}|^n |\mathbf{U}|^p$, this is

$$f(\mathbf{X}; \mathbf{M}, \mathbf{U}, \mathbf{V}) = (2\pi)^{-np/2} |\mathbf{U}|^{-p/2} |\mathbf{V}|^{-n/2} \exp\left\{ -\frac{1}{2} \mathrm{tr}\left[\mathbf{V}^{-1} (\mathbf{X} - \mathbf{M})^T \mathbf{U}^{-1} (\mathbf{X} - \mathbf{M}) \right] \right\}.$$

This is a nonlinear restriction on the natural parameter space of the full multivariate normal family. For the matrix variate normal, the covariance matrix is restricted to be defined as the Kronecker product of two covariance matrices. This makes its parameter space a lower-dimensional curved subset of the full multivariate normal distribution Roś et al. (2016).

Since $(cPhi) \otimes (c^{-1}\Phi) = \Phi \otimes \Sigma$, the parameters of Φ and Σ are not defined uniquely. We assume that $\psi_{qq} = 1$. As stated before, this makes the distribution a curved exponential family.

By differentiating the likelihood, we can obtain the MLEs for Φ and Σ.

$$\hat{\Phi} = \frac{1}{Np} \sum_{i=1}^{N} X_{ic}' \hat{\Sigma}^{-1} X_{ic}$$

$$\hat{\Sigma} = \frac{1}{Nq} \sum_{i=1}^{N} X_{ic} \hat{\Phi}^{-1} X_{ic}'$$

If $N > \max(p, q)$, then the flip-flop algorithm has only one solution and the MLEs are unique Srivastava et al. (2009).

For the tensor variate normal model with k modes, if we identify each precision matrix by setting the restriction $\det\Sigma_1^{1/d_1} = \det\Sigma_2^{1/d_2} \cdots = \det\Sigma_k^{1/d_k}$,then there are results about the convergence for each precision matrix. –>

For universal constants $C, c_1, c_2 > 0$ and suppose that $t \geq 1$ and $n \geq Ck^2 \frac{d_{\max}^3}{D} t^2$.

Then with probability at least $1 - ke^{-c_1 t^2 d_{\max}} - k^2 \left(\frac{\sqrt{nD}}{k d_{\max}} \right)^{-c_2 d_{\min}}$, the MLE $\hat{\Sigma}$ for n independent samples with precision matrix Σ is unique and satisfies Franks et al. (2026)

$$d_{\mathrm{FR}}(\hat{\Sigma}, \Sigma) = O\left(\frac{\sqrt{k} d_{\max}}{\sqrt{n}} t \right) \quad \text{and} \quad d_{\mathrm{FR}}(\hat{\Sigma}_a, \Sigma_a) = O\left(\frac{\sqrt{k d_a} d_{\max}}{\sqrt{nD}} t \right), \text{ for all } a \in [k]$$

–>

This ensures that

$$\mathrm{vec}(\mathcal{X}) \sim N\left(\mathrm{vec}(\mathcal{M}), {}_D \otimes \cdots \otimes {}_2 \otimes {}_1 \right)$$

Thus, the tensor variate normal is obtained by transforming a tensor of IID standard normal draws along each mode. If $\mathcal{X}$ has dimensions $n_1 \times \cdots \times n_D$, then

$$\text{vec}(\mathcal{X}) \sim N_{n^*}\left(\text{vec}(\mathcal{M}), \bigotimes_{d=1}^{D} \Sigma_d\right), \qquad n^* = \prod_{d=1}^{D} n_d.$$

The density function of the multilinear normal distribution is Ohlson et al. (2013)

$$f(\mathcal{X})(2\pi)^{-n^*/2}\left(\prod_{d=1}^{D}|\Sigma_d|^{-n^*/(2n_d)}\right)\exp\left\{-\frac{1}{2}\,\text{vec}(\mathcal{X}-\mathcal{M})^T\left(\bigotimes_{d=1}^{D}\Sigma_d^{-1}\right)\text{vec}(\mathcal{X}-\mathcal{M})\right\}.$$

Each Σ_d is positive definite and describes the covariance structure corresponding to mode d.

Maximum Likelihood Estimation Details

Parameter Initialization

The flip-flop and EM algorithms both require initial values for the parameters. For the flip-flop algorithm, the location tensor $\mathcal{M}$ is initialized as the mean of all of the draws.

$$\mathcal{M}^{(0)} = \bar{\mathcal{X}} = \frac{1}{N}\sum_{i=1}^{N}\mathcal{X}_i$$

To initialize the mode-specific covariance matrices, each centered observation is matricized along mode k. We pool the crossproducts of these matrices across all observations.

$$\tilde{\Sigma}_k^{(0)} = \frac{1}{NI_{-k}}\sum_{i=1}^{N}(\mathcal{X}_i - \mathcal{M}^{(0)})_{(k)}(\mathcal{X}_i - \mathcal{M}^{(0)})_{(k)}^T$$

where $I_{-k} = \prod_{\substack{d=1\\d\neq k}}^{D} I_d = \frac{I^*}{I_k}$.

It is then normalized so its trace is the size of mode k.

$$\Sigma_k^{(0)} = \frac{I_k}{\text{tr}(\tilde{\Sigma}_k^{(0)})}\tilde{\Sigma}_k^{(0)}$$

For the normal variance-mean mixture families, the covariance matrices are initialized using estimates obtained by fitting a tensor-variate normal model. Initial values for the remaining parameters are family specific.

The location tensor is initialized using the median. This provides a robust starting value that is less influenced by skewness.

$$\mathcal{M}^{(0)} = \tilde{\mathcal{X}}$$

Under the normal variance-mean mixture family,

$$E[\mathcal{X}] = \mathcal{M} + E[W]\mathcal{A}$$

We can use this expectation to initialize the skewness tensor $\mathcal{A}$.

$$\begin{aligned}\mathcal{A}^{(0)} &= \frac{1}{E_{\theta^{(0)}}[W]}(E[\mathcal{X}] - \mathcal{M}) \\ &= \frac{1}{E_{\theta^{(0)}}[W]}(\bar{\mathcal{X}} - \mathcal{M}^{(0)})\end{aligned}$$

Initial values for other parameters are chosen separately for each family. For example, for the skewed t, $\nu^{(0)} = 20$.

Flip-Flop Estimation

For the matrix variate normal model, differentiating the likelihood gives the updates

$$\hat{\mathbf{V}} = \frac{1}{Nn}\sum_{i=1}^{N}\mathbf{X}_{ic}^T\hat{\mathbf{U}}^{-1}\mathbf{X}_{ic}$$

and

$$\hat{\mathbf{U}} = \frac{1}{Np}\sum_{i=1}^{N}\mathbf{X}_{ic}\hat{\mathbf{V}}^{-1}\mathbf{X}_{ic}^T.$$

Each estimate depends on the other, which leads to the iterative flip-flop algorithm. If $N > \max(n, p)$, then the flip-flop algorithm has only one solution and the MLEs are unique under the conditions described by Srivastava et al. (2009).

Convergence Criterion

Convergence checks are done by comparing the change in likelihood relative to the previous iteration. At the t-th iteration, the equation used is

$$A = \frac{|\ell_t - \ell_{t-1}|}{|\ell_{t-1}| + 10^{-8}}$$

The algorithm ends when $A < \epsilon$, and by default $\epsilon = 1e-4$.

Rosenblatt Interpoint Test Derivations

Known-Parameter Null Distribution

They derive the null variances of $\bar{\delta}$ and $\bar{S}$ which are used to standardize the statistics.

$$\text{Var}(\bar{\delta}) = \frac{p(2N+3)}{90N(N-1)}$$

and

$$\text{Var}(\bar{S}) = \begin{cases} \frac{1}{\binom{N}{2}}\left(\frac{1}{56700}[989 + 202(N-2)]\right), & p = 2 \\ \frac{1}{\binom{N}{2}}\left(\frac{1}{1050}[37 + 6(N-2)]\right), & p = 3 \\ \frac{1}{\binom{N}{2}}\left(\frac{49p^2}{16200} + \frac{101p}{37800} + 2(N-2)\left[\frac{p^2}{16200} + \frac{29p}{37800}\right]\right), & p \geq 4 \end{cases}$$

Let $\sigma_{\bar{\delta}} = \sqrt{\text{Var}(\bar{\delta})}$ and $\sigma_{\bar{S}} = \sqrt{\text{Var}(\bar{S})}$.
Then with the Central Limit Theorem for U-processes, we can say that

$$\frac{\bar{\delta} - p/6}{\sigma_{\bar{\delta}}} \xrightarrow{D} N(0,1)$$

and

$$\frac{\bar{S} - 7p/180}{\sigma_{\bar{S}}} \xrightarrow{D} N(0,1)$$

They define

$$Q_1 = \frac{(\bar{\delta} - p/6)^2}{\mathrm{Var}(\bar{\delta})}$$

and

$$Q_2 = \frac{(\bar{S} - 7p/180)^2}{\mathrm{Var}(\bar{S})}$$

.

Then $Q_1 \xrightarrow{D} \chi^2_1$ and $Q_2 \xrightarrow{D} \chi^2_1$.

Adjustment for Estimated Parameters

Both the mean interpoint distance and its centered second moment are U-statistics of degree 2 because each kernel is averaged over all pairs of observations. Results for U-statistics with estimated nuisance parameters can be used to adjust their asymptotic variances Desgagné et al. (2025). Let $\mathbf{U}_i(\theta) = R_\theta(\mathbf{X}_i)$ represent the Rosenblatt transformation of observation $\mathbf{X}_i$. In practice, the transformation is evaluated at the MLE $\widehat{\theta}$. To derive the kernels of degree 1, let θ_0 denote the true parameter and write $\mathbf{U}_i = \mathbf{U}_i(\theta_0)$. Under the null model, these known-parameter transforms are independent Uniform($[0,1]^p$) random vectors. The subsequent score adjustment accounts for replacing θ_0 with $\widehat{\theta}$.

Following Assumption A2 of Desgagné et al. (2025), for a centered U-statistic of degree 2 with kernel $h(\mathbf{u}, \mathbf{v})$, define its kernel of degree 1 as

$$h^{(1)}(\mathbf{u}) = E\left[h(\mathbf{u}, \mathbf{V}) \mid \mathbf{u}\right],$$

where $\mathbf{V}$ is an independent Uniform($[0,1]^p$) random vector. Because the interpoint statistics have degree $\nu = 2$, Assumption A2 gives the expansion

$$\sqrt{N}\hat{T}_N(\theta_0) = \frac{2}{\sqrt{N}} \sum_{i=1}^{N} h^{(1)}(\mathbf{U}_i) + O_p(N^{-1/2}).$$

Consequently, the asymptotic variance of $\sqrt{N}\hat{T}_N(\theta_0)$ is $4\,\mathrm{Var}\{h^{(1)}(\mathbf{U})\}$. We denote the kernels of degree 1 associated with the two centered interpoint statistics by a_1 and a_2.

The centered mean-interpoint statistic is

$$\hat{\Delta}(\hat{\theta}) = \frac{1}{\binom{N}{2}} \sum_{i<j} \left(D_{ij} - \frac{p}{6}\right) = \bar{\delta} - \frac{p}{6}$$

with kernel

$$h_1(\mathbf{u},\mathbf{v}) = D(\mathbf{u},\mathbf{v}) - \frac{p}{6}, \qquad D(\mathbf{u},\mathbf{v}) = \sum_{r=1}^{p}(u_r - v_r)^2.$$

For $V_r \sim \text{Uniform}(0,1)$,

$$\begin{aligned} E\left[(u_r - V_r)^2 \mid u_r\right] &= u_r^2 - 2u_r E[V_r] + E[V_r^2] \\ &= u_r^2 - u_r + \frac{1}{3}. \end{aligned}$$

Also, $E[(U_r - V_r)^2] = 1/6$. Therefore, under the known-parameter null, the kernel of degree 1 associated with $\hat{\Delta}$ is

$$\begin{aligned} a_{1i} = h_1^{(1)}(\mathbf{U}_i) &= E\left[h_1(\mathbf{U}_i,\mathbf{V}) \mid \mathbf{U}_i\right] \\ &= \sum_{r=1}^{p}\left(U_{ir}^2 - U_{ir} + \frac{1}{6}\right). \end{aligned}$$

The centered second-moment statistic is

$$\hat{S}(\hat{\theta}) = \frac{1}{\binom{N}{2}}\sum_{i<j}\left\{\left(D_{ij} - \frac{p}{6}\right)^2 - \frac{7p}{180}\right\} = \bar{S} - \frac{7p}{180}.$$

The centering constant follows from

$$\text{Var}\left\{(U_r - V_r)^2\right\} = \frac{1}{15} - \frac{1}{36} = \frac{7}{180}.$$

Because the coordinate contributions are independent, the variance of their sum is $7p/180$.

Define

$$b_r(u_r) = u_r^2 - u_r + \frac{1}{3}$$

and

$$A(\mathbf{u}) = \sum_{r=1}^{p} b_r(u_r).$$

Then $E[D(\mathbf{u},\mathbf{V}) \mid \mathbf{u}] = A(\mathbf{u})$. In addition,

$$E[(u_r - V_r)^4 \mid u_r] = u_r^4 - 2u_r^3 + 2u_r^2 - u_r + \frac{1}{5},$$

and subtracting $b_r(u_r)^2$ gives

$$\operatorname{Var}\left\{(u_r - V_r)^2 \mid u_r\right\} = \frac{1}{3} b_r(u_r) - \frac{1}{45},$$

so independence across coordinates gives

$$E[D(\mathbf{u}, \mathbf{V})^2 \mid \mathbf{u}] = A(\mathbf{u})^2 + \frac{1}{3} A(\mathbf{u}) - \frac{p}{45}.$$

Let

$$A_i = A(\mathbf{U}_i) = \sum_{r=1}^{p} \left(U_{ir}^2 - U_{ir} + \frac{1}{3} \right).$$

The kernel of degree 1 associated with the centered second-moment statistic is therefore

$$\begin{aligned} a_{2i} = h_2^{(1)}(\mathbf{U}_i) &= E\left[\left(D(\mathbf{U}_i, \mathbf{V}) - \frac{p}{6} \right)^2 - \frac{7p}{180} \middle| \mathbf{U}_i \right] \\ &= A_i^2 + \frac{1-p}{3} A_i + \frac{p^2}{36} - \frac{11p}{180}. \end{aligned}$$

Thus, a_{1i} and a_{2i} are evaluations of the kernels of degree 1 associated with the centered U-statistics $\hat{\Delta}(\hat{\theta})$ and $\hat{S}(\hat{\theta})$, respectively. They are not kernels for the squared statistics Q_1 and Q_2; instead, their variances determine the denominators used to construct those statistics. The score adjustment below accounts for evaluating the Rosenblatt transformation at $\hat{\theta}$ rather than θ_0.

Let

$$\mathbf{s}_i = \frac{\partial}{\partial \theta} \log f(\mathbf{X}_i; \theta) \Big|_{\theta = \hat{\theta}}$$

be the fitted-model score. The parameter vector θ contains the MLEs obtained from the flip-flop or EM algorithm.

For $m \in \{1, 2\}$, let $\bar{a}_m$ denote the sample mean of a_{mi} and $\bar{\mathbf{s}}$ denote the sample mean score vector. We estimate the variance of the kernel of degree 1, the score information matrix, and their cross-covariance.

$$\hat{H}_m = \frac{1}{N}\sum_{i=1}^{N}(a_{mi} - \bar{a}_m)^2$$
$$\hat{G}_m = \frac{1}{N}\sum_{i=1}^{N}(a_{mi} - \bar{a}_m)(\mathbf{s}_i - \bar{\mathbf{s}})^T$$
$$\hat{I} = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{s}_i - \bar{\mathbf{s}})(\mathbf{s}_i - \bar{\mathbf{s}})^T$$

Then, the fitted-parameter-adjusted asymptotic variance is

$$\hat{\Omega}_{m,\text{adj}} = 4(\hat{H}_m - \hat{G}_m \hat{I}^{+} \hat{G}_m^T)$$

where $\hat{I}^{+}$ is the Moore-Penrose pseudoinverse.

Bader, Brett W., and Tamara G. Kolda. 2006. "Algorithm 862: MATLAB Tensor Classes for Fast Algorithm Prototyping." *ACM Transactions on Mathematical Software* 32 (4): 635–53. https://doi.org/10.1145/1186785.1186794.

Bader, Brett W., and Tamara G. Kolda. 2007. "Efficient MATLAB Computations with Sparse and Factored Tensors." *SIAM Journal on Scientific Computing* 30 (1): 205–31. https://doi.org/10.1137/060676489.

Bader, Brett W., Tamara G. Kolda, et al. 2025. *Tensor Toolbox for MATLAB*. https://www.tensortoolbox.org/.

Barndorff-Nielsen, O., J. Kent, and M. Sørensen. 1982. "Normal Variance-Mean Mixtures and z Distributions." *International Statistical Review* 50 (2): 145–59.

Carroll, J. Douglas, and Jih-Jie Chang. 1970. "Analysis of Individual Differences in Multidimensional Scaling via an n-Way Generalization of Eckart-Young Decomposition." *Psychometrika* 35 (3): 283–319. https://doi.org/10.1007/BF02310791.

De Lathauwer, Lieven, Bart De Moor, and Joos Vandewalle. 2000. "A Multilinear Singular Value Decomposition." *SIAM Journal on Matrix Analysis and Applications* 21: 1253–78. https://doi.org/10.1137/S0895479896305696.

Desgagné, Alain, Christian Genest, and Frédéric Ouimet. 2025. "Asymptotics for Non-Degenerate Multivariate u-Statistics with Estimated Nuisance Parameters Under the Null and Local Alternative Hypotheses." *Journal of Multivariate Analysis* 208: 105398. https://doi.org/https://doi.org/10.1016/j.jmva.2024.105398.

Franks, Cole, Rafael Oliveira, Akshay Ramachandran, and Michael Walter. 2026. "Near Optimal Sample Complexity for Matrix and Tensor Normal Models via Geodesic Convexity." *The Annals of Statistics* 54 (1). https://doi.org/10.1214/25-aos2539.

Harshman, R. A. 1970. *Foundations of the PARAFAC Procedure: Models and Conditions for an Explanatory Multimodal Factor Analysis*.

Hitchcock, Frank L. 1927. "The Expression of a Tensor or a Polyadic as a Sum of Products." *Journal of Mathematics and Physics* 6 (1-4): 164–89. https://doi.org/10.1002/sapm192761164.

Kolda, Tamara G., and Brett W. Bader. 2009. "Tensor Decompositions and Applications." *SIAM Review* 51 (3): 455–500. https://doi.org/10.1137/07070111X.

Kossaifi, Jean, Yannis Panagakis, Anima Anandkumar, and Maja Pantic. 2019. "TensorLy: Tensor Learning in Python." *Journal of Machine Learning Research* 20 (26): 1–6. https://jmlr.org/papers/v20/18-277.html.

Li, Jing, Jacob Bien, and Martin T. Wells. 2024. *rTensor: Tools for Tensor Analysis and Decomposition*. https://cran.r-project.org/package=rTensor.

Li, Jing, Jacob Bien, Martin T. Wells, Han Xiao, and Yuan Chen. 2024. *ttTensor: Tensor Train Decomposition*. https://cran.r-project.org/package=ttTensor.

Ohlson, Martin, M. Rauf Ahmad, and Dietrich von Rosen. 2013. "The Multilinear Normal Distribution: Introduction and Some Basic Properties." *Journal of Multivariate Analysis* 113: 37–47. https://doi.org/10.1016/j.jmva.2011.05.015.

Oseledets, I. V. 2011. "Tensor-Train Decomposition." *SIAM Journal on Scientific Computing* 33 (5): 2295–317. https://doi.org/10.1137/090752286.

Periša, Lana. 2023. *TensorToolbox.jl: Tensor Decompositions in Julia*. https://github.com/lanaperisa/TensorToolbox.jl.

pyttb developers. 2026. *pyttb: Python Tensor Toolbox*. https://pyttb.readthedocs.io/.

Roś, Beata, Fetsje Bijma, Jan C. de Munck, and Mathisca C. M. de Gunst. 2016. "Existence and Uniqueness of the Maximum Likelihood Estimator for Models with a Kronecker Product Covariance Structure." *Journal of Multivariate Analysis* 143: 345–61. https://doi.org/10.1016/j.jmva.2015.05.019.

Rosenblatt, Murray. 1952. "Remarks on a Multivariate Transformation." *The Annals of Mathematical Statistics* 23 (3): 470–72. http://www.jstor.org/stable/2236692.

Schürholt, Konstantin, D. Taskiran, B. Knyazev, X. Giró-i-Nieto, and D. Borth. 2022. "Model Zoo: A Dataset of Diverse Populations of ResNet-18 Models – CIFAR-10." Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.6974029.

Srivastava, Muni S., Tatjana von Rosen, and Dietrich von Rosen. 2009. "Estimation and Testing in General Multivariate Linear Models with Kronecker Product Covariance Structure." *Sankhyā: The Indian Journal of Statistics, Series A* 71 (2): 137–63.

TensorOperations.jl developers. 2026. *TensorOperations.jl*. https://quantumkithub.github.io/TensorOperations.jl/stable/.

Yang, Mengta, and Reza Modarres. 2017. "Multivariate Tests of Uniformity." *Statistical Papers* 58 (September). https://doi.org/10.1007/s00362-015-0715-x.